INFORME DE DESARROLLO

PARCIAL 2

Sistemas Operativos

Implementación de un servidor Ubuntu, acceso remoto seguro, transferencia de archivos y clasificación automatizada con Bash y Ollama

Estudiante: Johan David Baracaldo Caicedo

Entorno servidor: VM Ubuntu

Sistema operativo servidor: Ubuntu Server 22.04.5 LTS

Cliente de administración: Windows PowerShell

Fecha de evidencias: 17 de octubre de 2026

Resumen ejecutivo

Este informe documenta la puesta en marcha y validación de un entorno de administración remota para una instancia Ubuntu Server alojada en una VM. El trabajo integra operaciones fundamentales de sistemas operativos y administración de servidores: creación de una estructura de directorios, preparación de archivos de prueba en un equipo Windows, transferencia mediante SCP, verificación del servicio SSH, configuración de reglas de firewall con UFW, instalación y comprobación de Ollama, ejecución de un modelo de lenguaje local y desarrollo de un script Bash para clasificar archivos por extensión.

Las capturas aportadas evidencian la creación de los directorios del proyecto, la existencia de cuatro archivos de prueba, la transferencia de estos al servidor y su clasificación final en dos categorías: documentos y trabajo. También se observa que los servicios SSH y Ollama reportan estado activo, y que UFW está habilitado con política predeterminada de denegar conexiones entrantes y permitir las salientes.

El informe distingue entre los resultados que se observan directamente y las recomendaciones que conviene aplicar antes de considerar el entorno terminado. En particular, se recomienda revisar la salida del resumen de conteo del script, confirmar que todas las reglas de firewall sean necesarias y restringir los puertos de administración o de aplicación a los orígenes autorizados. La evidencia del modelo local corresponde a una respuesta generada con límite de salida, por lo que se describe como una prueba funcional y no como una evaluación exhaustiva de calidad.

Palabras clave

Ubuntu Server, SSH, SCP, UFW, Bash, automatización, Ollama, modelo de lenguaje local, administración de archivos, seguridad informática.

Tabla de contenido

- 1. Introducción
- 2. Objetivos
- 3. Alcance y metodología

- 4. Marco conceptual
- 5. Entorno de trabajo
- 6. Desarrollo de la práctica
- 7. Resultados y análisis
- 8. Seguridad y buenas prácticas
- 9. Limitaciones y oportunidades de mejora
- 10. Conclusiones
- 11. Recomendaciones finales
- Anexo A. Comandos de referencia
- Anexo B. Evidencias fotográficas

1. Introducción

La administración de sistemas operativos comprende la instalación y configuración de servicios, el control de acceso, la gestión de archivos, la automatización de tareas y la verificación del estado de los recursos. En un entorno de nube, estas actividades se realizan frecuentemente de manera remota, por lo que la conectividad segura y la aplicación de reglas de firewall son componentes esenciales.

La práctica desarrollada utiliza una instancia Ubuntu Server en OCI y un equipo Windows como cliente de administración. Desde PowerShell se prepara un conjunto pequeño de archivos representativos y se transfieren al servidor. En el sistema Linux se crea una estructura de carpetas para separar documentos, imágenes, archivos de trabajo, comprimidos y otros elementos. Posteriormente, se instala o comprueba el funcionamiento de Ollama y se consulta un modelo de lenguaje local para obtener una propuesta general de organización de archivos.

Finalmente, un script de Bash procesa los archivos que se encuentran en la carpeta de entrada, identifica su extensión y los mueve a una carpeta de destino. El listado final permite comprobar la ubicación de los archivos. Las capturas constituyen el soporte visual de las acciones y resultados descritos en este documento.

2. Objetivos

2.1 Objetivo general

Implementar y documentar un entorno básico de administración remota de un servidor Ubuntu, incorporando transferencia de archivos, controles de acceso de red, ejecución de un modelo de inteligencia artificial local y automatización de la clasificación de archivos mediante Bash.

2.2 Objetivos específicos

- Crear en el servidor una estructura de directorios organizada para recibir y clasificar archivos.
- Preparar archivos de prueba desde Windows PowerShell y transferirlos al servidor mediante SCP.
- Comprobar el estado del servicio SSH y verificar que la administración remota sea posible.

- Revisar la configuración activa de UFW y describir el alcance de las reglas observadas.
- Comprobar que Ollama se encuentre activo y que el modelo qwen2.5:0.5b-instruct esté disponible.
- Enviar una solicitud al modelo local y analizar su respuesta como apoyo a la definición de categorías de archivos.
- Ejecutar un script Bash que clasifique archivos según su extensión y verificar sus ubicaciones finales.
- Identificar aspectos de seguridad y validación que permitan mejorar la confiabilidad de la solución.

3. Alcance y metodología

El informe se construye a partir del documento de evidencias proporcionado, compuesto por nueve capturas de terminal. Se describen únicamente las acciones y estados que pueden inferirse razonablemente de ellas. Cuando una captura no permite comprobar un detalle —por ejemplo, el contenido completo del script, el origen exacto de cada regla de firewall o el resultado de una transferencia de regreso al cliente— se presenta como limitación y no como un hecho confirmado.

La metodología fue secuencial: preparación del espacio de trabajo, creación de archivos en el cliente, transferencia, verificación en el servidor, comprobación de servicios, consulta al modelo local, automatización con Bash y validación final. Las recomendaciones técnicas incluidas al final son propuestas para reforzar la práctica; no se presentan como acciones ya ejecutadas.

4. Marco conceptual

4.1 Ubuntu Server y administración remota

Ubuntu Server es una distribución GNU/Linux utilizada para alojar servicios y ejecutar tareas administrativas. En un servidor remoto, las operaciones suelen realizarse a través de una consola segura, sin depender de una interfaz gráfica. La línea de comandos permite administrar directorios, permisos, servicios y procesos con precisión, siempre que los comandos se ejecuten con el usuario y los privilegios adecuados.

4.2 SSH

Secure Shell (SSH) proporciona una sesión cifrada para autenticarse y ejecutar comandos en un equipo remoto. En las evidencias, el comando ssh conecta desde Windows al usuario ubuntu de la instancia. La salida de whoami identifica el usuario efectivo como ubuntu, hostname permite identificar el equipo remoto y systemctl is-active ssh comprueba el estado del servicio.

4.3 SCP y transferencia de archivos

SCP permite copiar archivos entre equipos utilizando la infraestructura de autenticación y cifrado de SSH. La opción -i indica el archivo de clave privada que el cliente utiliza para autenticarse. En la evidencia se observa un primer intento fallido con el mensaje “You must specify a subsystem to invoke”; después, la transferencia de los cuatro archivos aparece completada al 100 %. El resultado posterior del comando ls en el servidor confirma la presencia de configuracion.conf, datos.csv, informe.txt y script.sh en la carpeta de entrada.

4.4 UFW y filtrado de tráfico

Uncomplicated Firewall (UFW) es una interfaz de administración de reglas de filtrado en Ubuntu. La salida documentada muestra UFW activo, registros en nivel bajo, tráfico entrante denegado por defecto, tráfico saliente permitido y tráfico reenviado deshabilitado. También aparecen reglas de entrada para los puertos TCP 4100, 3000 y 22; SSH figura como OpenSSH en el puerto 22. Una regla abierta a Anywhere permite conexiones desde cualquier dirección de origen, por lo que su conveniencia debe evaluarse según el servicio que escuche en el puerto.

4.5 Ollama y modelos de lenguaje locales

Ollama permite descargar y ejecutar modelos de lenguaje en un equipo o servidor compatible. La captura muestra que el servicio se encuentra activo y que está instalado qwen2.5:0.5b-instruct, con tamaño aproximado de 397 MB según ollama list. La petición se realiza contra 127.0.0.1:11434, lo que indica que la consulta se dirige a la interfaz local del servidor y no requiere exponer directamente la API a Internet.

4.6 Bash y clasificación de archivos

Bash es un intérprete de comandos y lenguaje de scripting habitual en Linux. Un script puede automatizar tareas repetitivas, como crear carpetas, recorrer archivos, extraer extensiones y mover cada elemento a una ubicación de destino. Para evitar errores, el script debe tratar nombres con espacios, no sobrescribir archivos existentes sin autorización, manejar extensiones desconocidas y producir un resumen consistente con el contenido real de las carpetas.

5. Entorno de trabajo

Elemento	Detalle evidenciado
VM	Ubuntu VM
Sistema del servidor	Ubuntu Server 22.04.5 LTS;
Nombre de host	instance-20260707-1651.
Usuario remoto observado	ubuntu.
Cliente	Windows PowerShell, en C:\Users\johan\Documents\ParcialSO.
Acceso remoto	SSH mediante una clave privada indicada con la variable $key y la dirección almacenada en $ip.
Transferencia	SCP desde Windows a /home/ubuntu/parcial-so/archivos/entrada/.
Firewall	UFW activo; política entrante predeterminada deny y saliente allow.
IA local	Ollama activo; modelo qwen2.5:0.5b-instruct disponible.
Archivos de prueba	informe.txt, configuracion.conf, script.sh y datos.csv.

6. Desarrollo de la práctica

6.1 Creación de la estructura de directorios en Ubuntu

La primera evidencia muestra la creación de una estructura de trabajo bajo /home/ubuntu/parcial-so. Dentro de ella se organizan los directorios archivos, scripts y evidencias. A su vez, archivos contiene las carpetas entrada, documentos, imagenes, trabajo, comprimidos y otros. La carpeta entrada actúa como punto de recepción; documentos e imagenes separan contenidos comunes; trabajo reúne archivos relacionados con scripts, configuración o datos; comprimidos se reserva para archivos empaquetados; y otros permite conservar elementos que no encajan en las categorías previstas.

```
        mkdir -p /home/ubuntu/parcial-
so/{archivos/{entrada,documentos,imagenes,trabajo,comprimidos,otros},scripts,evidencias}
        chown -R ubuntu:ubuntu /home/ubuntu/parcial-so
        find /home/ubuntu/parcial-so -maxdepth 3 -type d | sort
```

La opción -p crea directorios intermedios y evita que la operación falle simplemente porque parte de la ruta ya exista. El cambio de propietario facilita que el usuario ubuntu administre el árbol sin tener que ejecutar todas las operaciones como root. El comando find con -type d enumera directorios; -maxdepth limita la profundidad y sort ordena la salida.

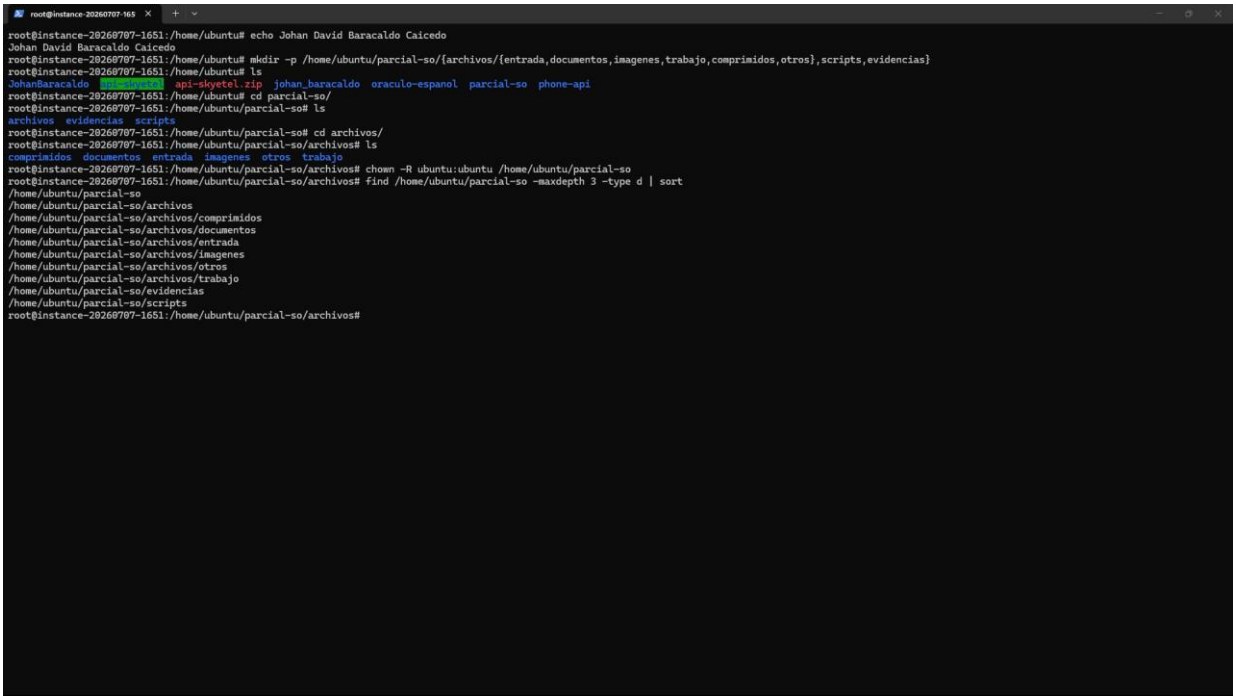


Figura 1. Creación y comprobación de la estructura de carpetas en el servidor.

6.2 Preparación de archivos desde Windows

En el cliente se creó la carpeta C:\Users\johan\Documents\ParcialSO y se generaron cuatro archivos de prueba. Cada archivo representa un tipo de contenido diferente: un texto descriptivo, una configuración de servicio, un script de shell y una tabla de datos en formato CSV. La salida de PowerShell muestra los nombres y tamaños de los cuatro elementos.

Archivo	Tipo	Uso en la práctica
informe.txt	Texto plano	Representa documentación.
configuracion.conf	Configuración	Representa parámetros de un servicio.
script.sh	Script Bash	Representa automatización ejecutable.
datos.csv	Datos tabulares	Representa información delimitada por comas.

La creación de estos archivos proporciona un conjunto de prueba controlado y pequeño. Esto facilita comprobar la transferencia y observar el comportamiento del clasificador sin involucrar archivos personales o información productiva.

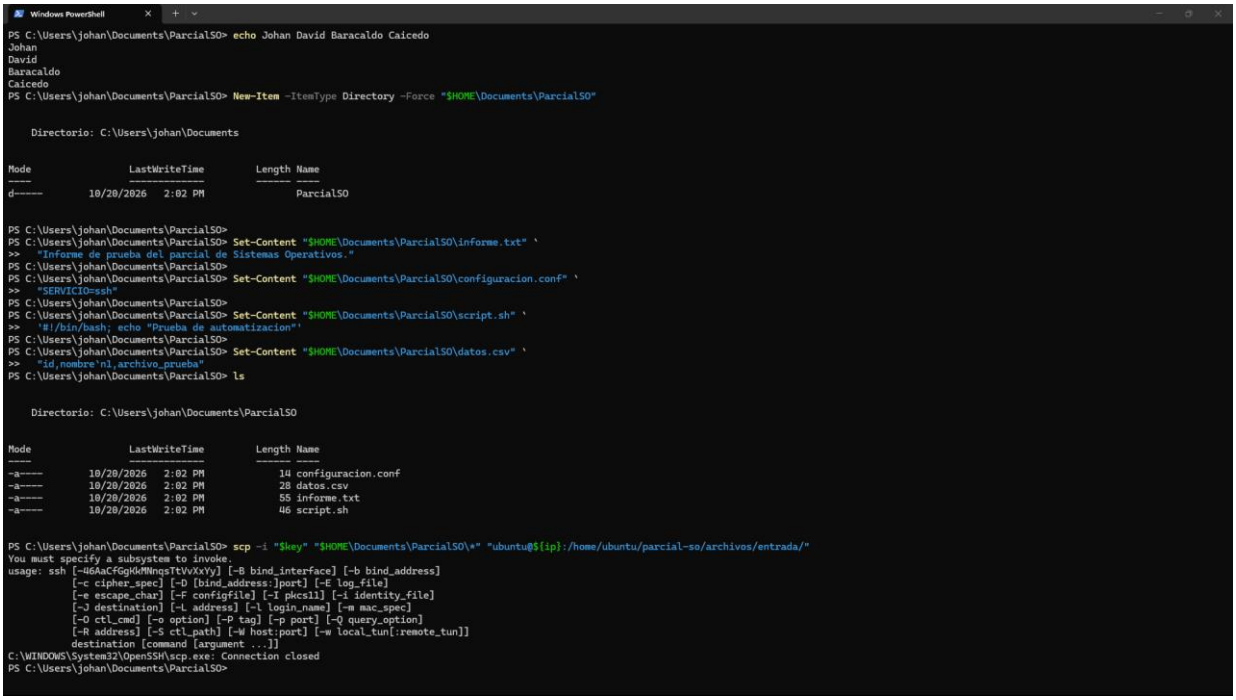


Figura 2. Creación de los cuatro archivos de prueba y primer intento de transferencia SCP.

6.3 Transferencia de archivos mediante SCP

La primera tentativa de copia terminó con el mensaje “You must specify a subsystem to invoke” y el cierre de la conexión. La captura posterior muestra una transferencia de cuatro archivos al 100 %, con sus nombres y tamaños, por lo que el intento posterior sí completó la copia. La evidencia no muestra con suficiente claridad todos los parámetros del comando corregido; por ello, el informe registra el resultado y no atribuye la corrección a una opción concreta que no pueda confirmarse.

El procedimiento utiliza una clave privada mediante -i y el destino remoto /home/ubuntu/parcial-so/archivos/entrada/. En Windows PowerShell conviene comprobar que las variables de ruta y dirección estén definidas correctamente, usar comillas rectas y validar el destino antes de iniciar la transferencia.

```
# Ejemplo de referencia en PowerShell; reemplazar rutas y dirección por los valores reales:
$key = "$HOME\Downloads\clave-privada.key"
$ip = "IP_PUBLICA_DEL_SERVIDOR"
scp -i "$key" .\* "ubuntu@${ip}:/home/ubuntu/parcial-so/archivos/entrada/"


# Si el cliente y el servidor tienen una incompatibilidad con el modo SCP basado en SFTP:
```

```
scp -O -i "$key" .\* "ubuntu@${ip}:/home/ubuntu/parcial-so/archivos/entrada/"
```

La opción -O fuerza el protocolo SCP tradicional en versiones recientes del cliente OpenSSH. Debe emplearse como alternativa de diagnóstico cuando el servidor o su configuración no responden correctamente al modo de transferencia basado en SFTP. No se deben compartir ni pegar claves privadas en el informe.

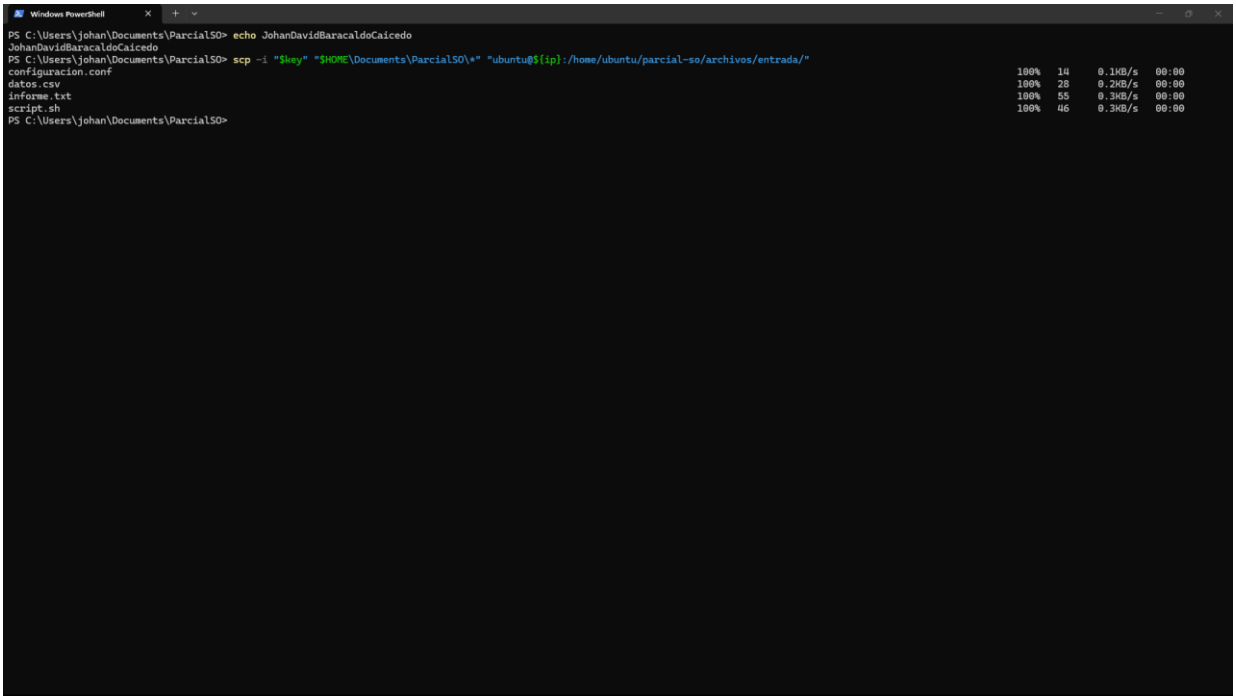


Figura 3. La transferencia de los cuatro archivos finaliza al 100 % en el cliente Windows.

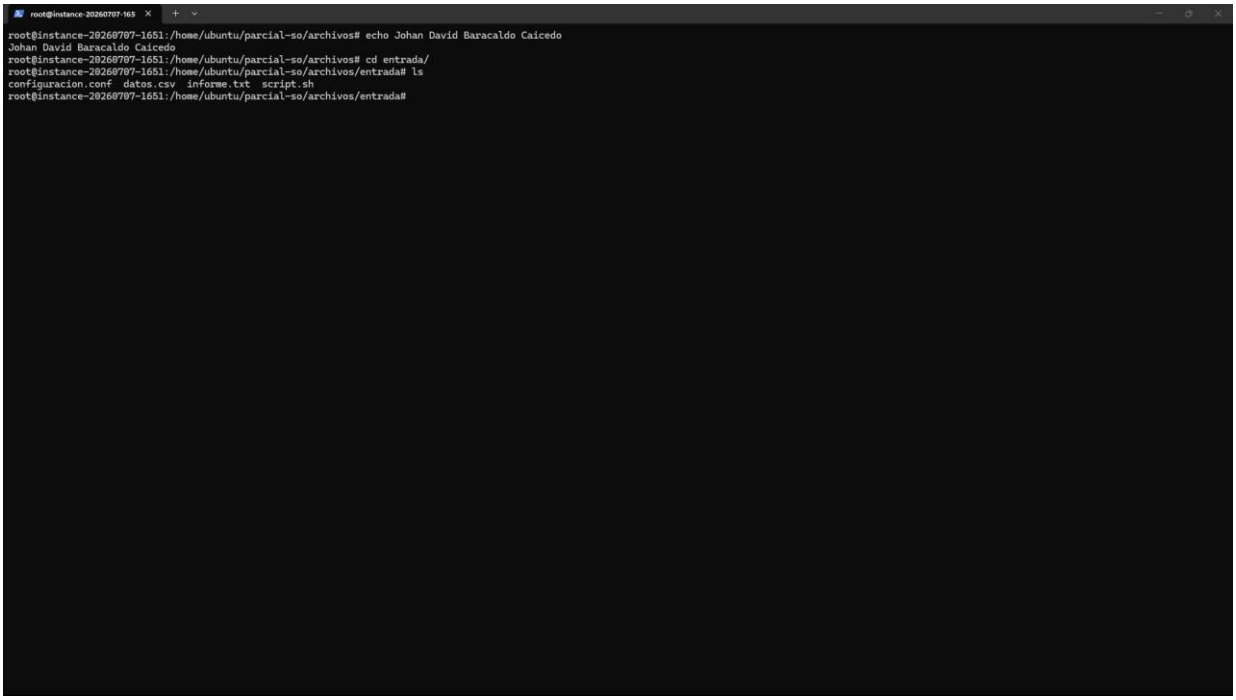


Figura 4. Verificación en Ubuntu de los archivos recibidos en la carpeta entrada.

6.4 Verificación de Ollama y consulta al modelo local

La captura muestra el comando systemctl is-active ollama con resultado active y la salida de ollama list, donde figura qwen2.5:0.5b-instruct, tamaño aproximado 397 MB. Esto permite concluir que el servicio estaba activo y que el modelo estaba disponible en el momento de la captura.

Posteriormente se ejecutó una petición HTTP a la API local de Ollama, dirigida a 127.0.0.1:11434/api/generate. La instrucción pidió al modelo proponer una estructura para clasificar archivos TXT, PDF, JPG, SH, CONF, CSV y ZIP, y explicar precauciones para evitar pérdidas. La respuesta visible sugiere categorías generales, pero se encuentra truncada debido al límite de generación configurado (num_predict: 120). Por tanto, la prueba confirma la comunicación con la API y la generación de una respuesta, pero no demuestra que el modelo haya producido una política completa o suficientemente precisa para ejecutar una clasificación real sin supervisión.

La consulta se envió al endpoint de loopback 127.0.0.1, que normalmente restringe la petición a la propia máquina. Es una elección prudente para una prueba local: no es necesario abrir el puerto 11434 al exterior para usar el modelo desde el servidor.

```
sudo systemctl is-active ollama
ollama list



# Ejemplo conceptual de consulta local (modelo y prompt pueden ajustarse):
curl http://127.0.0.1:11434/api/generate \
  -H 'Content-Type: application/json' \
  -d '{
    "model": "qwen2.5:0.5b-instruct",
    "prompt": "Propón categorías para organizar archivos por extensión. Indica precauciones para no
perder información.",
    "stream": false,
    "options": {"num_ctx": 1024, "num_predict": 120}
  }'
```

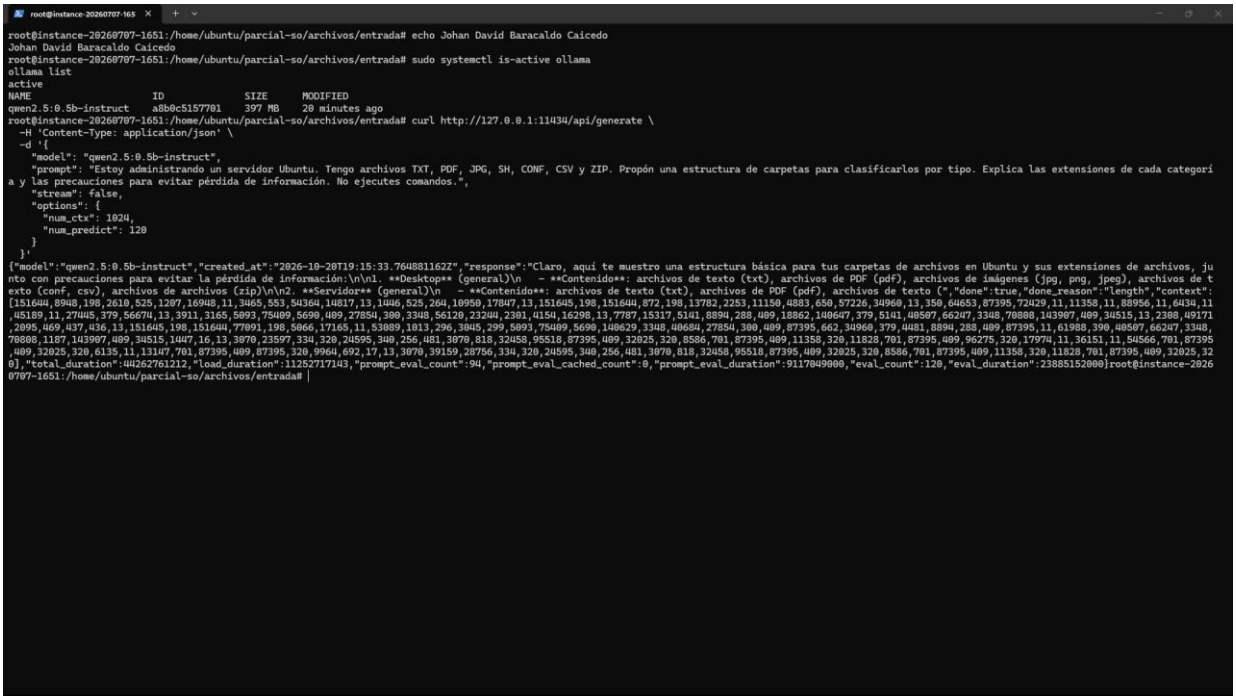


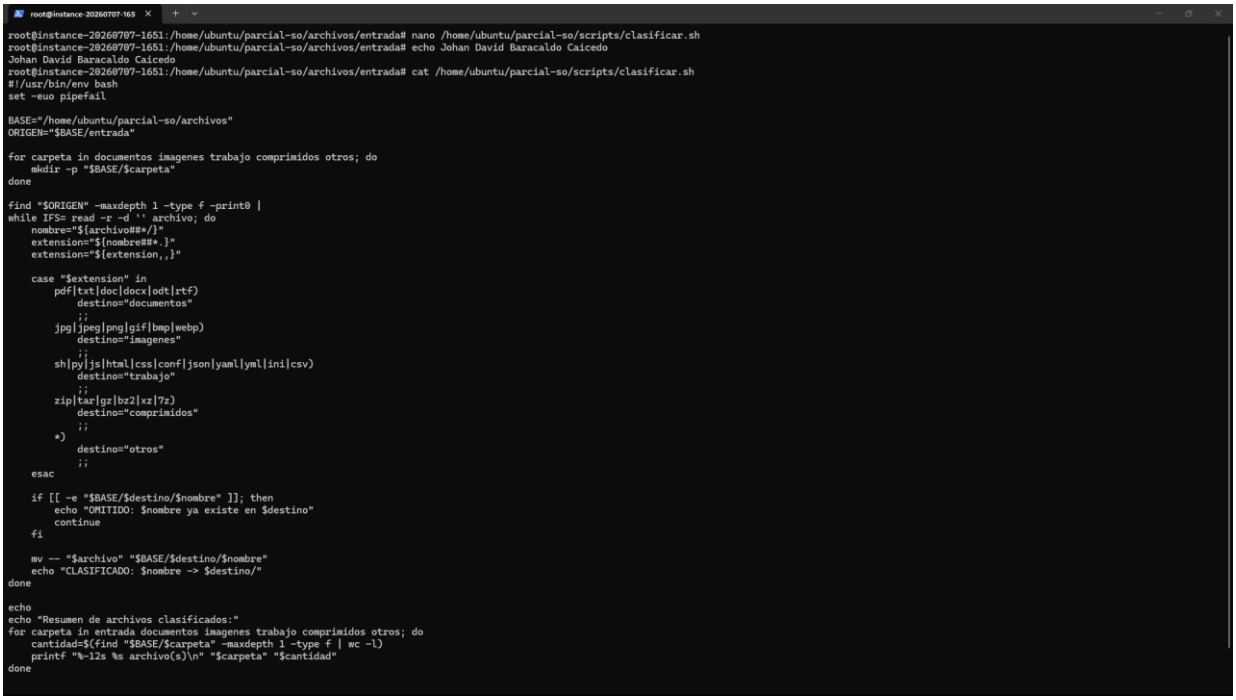
Figura 5. Estado activo de Ollama, modelo descargado y petición a la API local.

6.5 Desarrollo del script Bash de clasificación

La evidencia siguiente presenta el archivo /home/ubuntu/parcial-so/scripts/clasificar.sh. Se observa una estructura de script que contempla una ruta base, una carpeta de origen, la creación de directorios de destino y un bloque case que asigna extensiones a categorías. Entre las categorías que aparecen en la ejecución están documentos, imagenes, trabajo, comprimidos y otros.

El flujo lógico esperado es: identificar el directorio del proyecto; asegurar que existan las carpetas de destino; recorrer los archivos regulares de la carpeta entrada; obtener la extensión de cada nombre; convertirla a minúsculas; seleccionar el destino mediante reglas; evitar sobrescribir un archivo si ya existe otro con el mismo nombre; mover el archivo; y finalmente contar los archivos por carpeta. El script automatiza una tarea repetitiva y reduce la necesidad de mover manualmente cada archivo.

La captura de código es una imagen de terminal y parte del texto no es legible con suficiente precisión. Por ello, el informe explica el propósito y el comportamiento observado, sin presentar una transcripción literal del script que pudiera introducir errores.



```
root@instance-20260707-165:/home/ubuntu/parcial-so/archivos/entrada# nano /home/ubuntu/parcial-so/scripts/clasificar.sh
root@instance-20260707-165:/home/ubuntu/parcial-so/archivos/entrada# echo Johan David Baracaldo Caicedo
Johan David Baracaldo Caicedo
root@instance-20260707-165:/home/ubuntu/parcial-so/archivos/entrada# cat /home/ubuntu/parcial-so/scripts/clasificar.sh
#!/usr/bin/env bash
set -euo pipefail

BASE="/home/ubuntu/parcial-so/archivos"
ORIGEN="$BASE/entrada"

for carpeta in documentos imagenes trabajo comprimidos otros; do
    mkdir -p "$BASE/$carpeta"
done

find "$ORIGEN" -maxdepth 1 -type f -print0 |
while IFS= read -r -d '' archivo; do
    nombre="${archivo##*/}"
    extension="${nombre##*.}"
    extension="${extension,,}"

    case "$extension" in
        pdf|txt|doc|docx|odt|rtf)
            destino="documentos"
            ;;
        jpg|jpeg|png|gif|bmp|webp)
            destino="imagenes"
            ;;
        sh|py|js|html|css|conf|json|yaml|yml|ini|csv)
            destino="trabajo"
            ;;
        zip|tar|gz|bz2|xz|7z)
            destino="comprimidos"
            ;;
        *)
            destino="otros"
            ;;
    esac

    if [[ -e "$BASE/$destino/$nombre" ]]; then
        echo "OMITIDO: $nombre ya existe en $destino"
        continue
    fi

    mv -- "$archivo" "$BASE/$destino/$nombre"
    echo "CLASIFICADO: $nombre -> $destino/"
done

echo
echo "Resumen de archivos clasificados:"
for carpeta in entrada documentos imagenes trabajo comprimidos otros; do
    cantidad=$(find "$BASE/$carpeta" -maxdepth 1 -type f | wc -l)
    printf "%-12s %s archivo(s)\n" "$carpeta" "$cantidad"
done
```

Figura 6. Contenido del script Bash de clasificación automática.

6.6 Ejecución del clasificador y comprobación de resultados

En la ejecución se observan estos resultados: datos.csv se clasifica en trabajo; informe.txt se clasifica en documentos; configuracion.conf se clasifica en trabajo; y script.sh se clasifica en trabajo. El comando find posterior lista cuatro archivos y confirma que el informe de texto quedó en documentos y los otros tres elementos quedaron en trabajo.

El resultado es coherente con una regla que trata TXT como documento y CSV, CONF y SH como archivos de trabajo. No se observan archivos en imagenes ni comprimidos, porque el conjunto de prueba no contiene JPG, PDF o ZIP. Esto no significa que esas categorías estén mal definidas; únicamente indica que no fueron ejercitadas por los archivos presentes.

En la captura, el resumen impreso por el script parece mostrar un conteo que no coincide de forma clara con el listado final. La comprobación independiente con find es más fiable para verificar los cuatro archivos visibles, por lo que se recomienda corregir o volver a ejecutar el bloque de conteo antes de entregar la versión definitiva. También conviene comprobar que los archivos de entrada queden vacíos después de una clasificación completa.

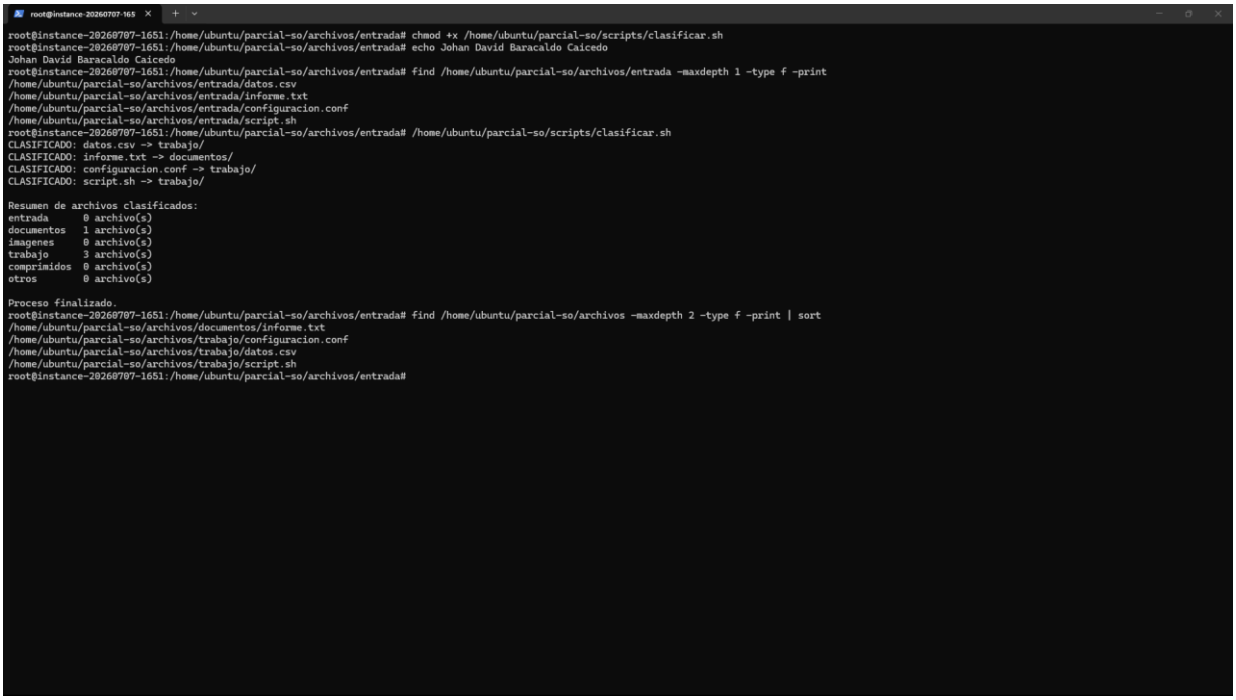


Figura 7. Ejecución del script y listado final de los archivos clasificados.

6.7 Validación desde el cliente remoto y revisión de firewall

La última parte de las evidencias muestra una nueva conexión SSH desde PowerShell al servidor. Dentro de la sesión se ejecutan whoami, hostname, systemctl is-active ssh, systemctl is-active ollama y sudo ufw status verbose. La salida confirma el usuario ubuntu, el host instance-20260707-1651, los servicios SSH y Ollama activos y UFW habilitado.

La configuración de UFW observada es: entrada denegada por defecto, salida permitida por defecto y tráfico reenviado deshabilitado. Aparecen reglas para TCP 4100, TCP 3000 y TCP 22, además de reglas IPv6 para 4100 y 3000. La salida visible no muestra una regla IPv6 equivalente para 22, por lo que conviene revisar el conjunto completo de reglas y las necesidades de conectividad IPv4/IPv6.

La validación demuestra que el acceso SSH seguía funcionando durante la comprobación y que la política del firewall no bloqueó esa sesión. Sin embargo, el hecho de que un puerto aparezca permitido no demuestra por sí mismo que exista un servicio seguro escuchando en él; los puertos 3000 y 4100 deben asociarse a aplicaciones concretas y restringirse si no necesitan acceso público.

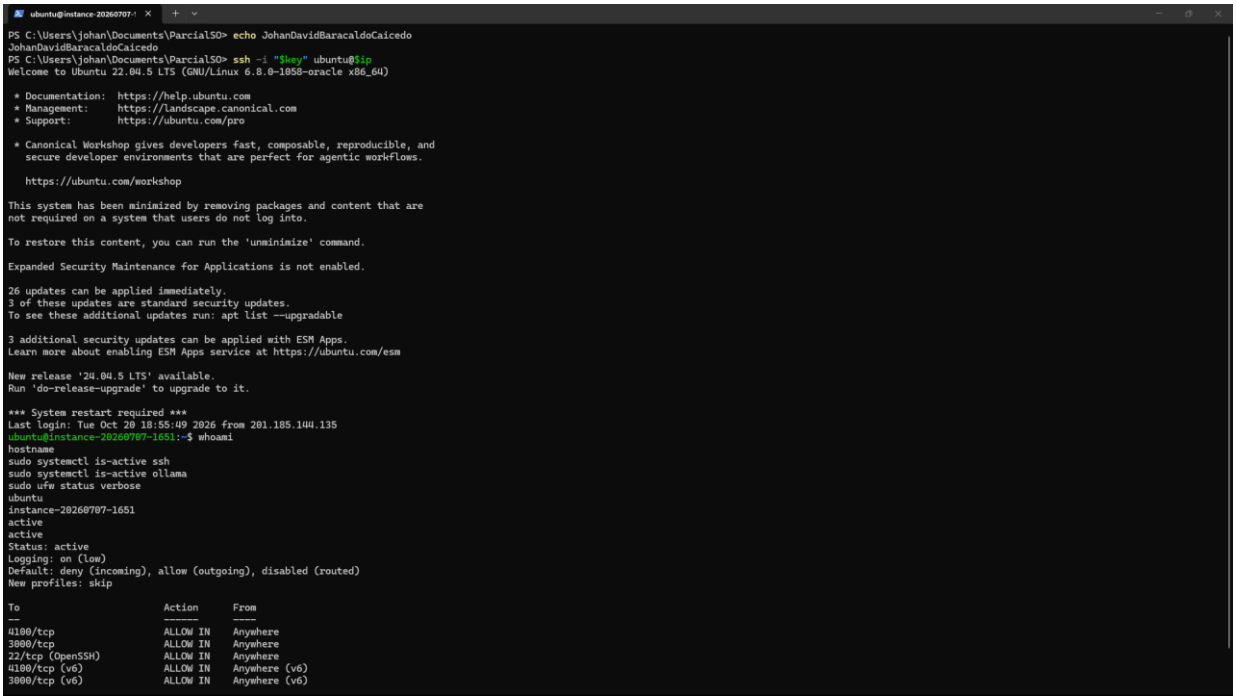


Figura 8. Acceso SSH desde Windows y comprobación de servicios y reglas UFW.

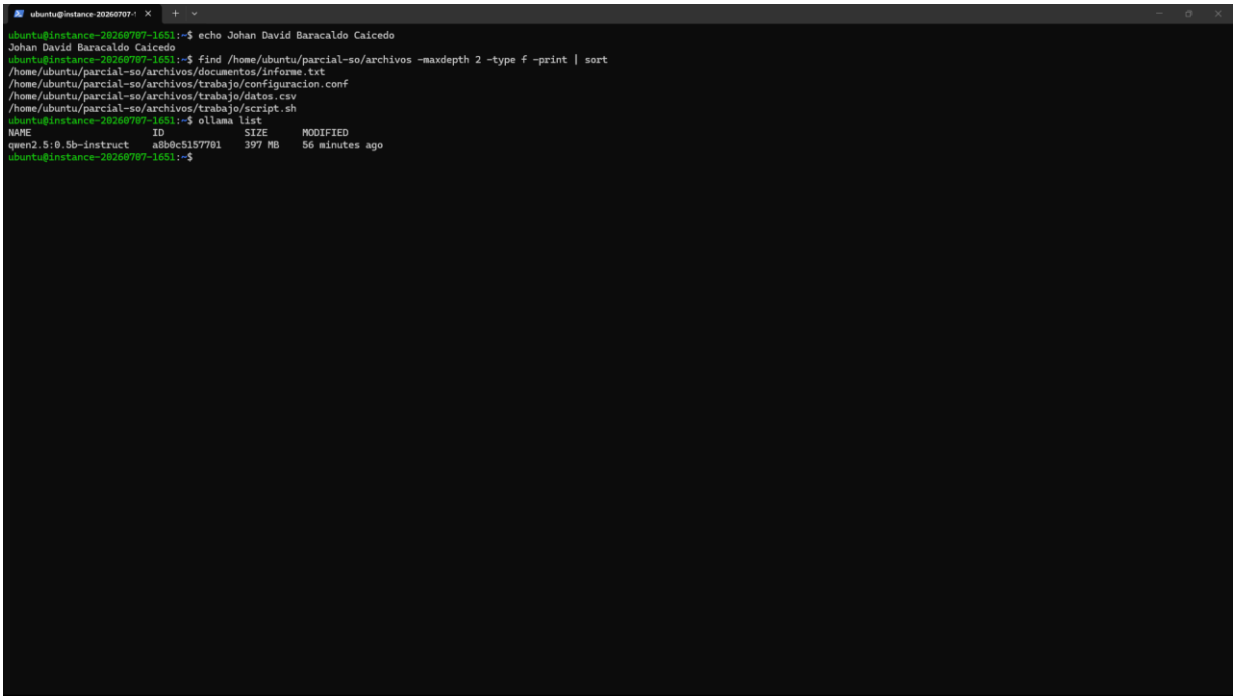


Figura 9. Comprobación final de la clasificación y disponibilidad del modelo local.

7. Resultados y análisis

7.1 Resultados verificables

- Se creó el árbol de directorios parcial-so en el servidor, con carpetas destinadas a la recepción, clasificación, scripts y evidencias.
- Se prepararon cuatro archivos de prueba en Windows: informe.txt, configuracion.conf, script.sh y datos.csv.
- Una transferencia posterior mediante SCP completó los cuatro archivos y el servidor los mostró en la carpeta entrada.
- El servicio SSH aparece activo y se comprobó una sesión remota con el usuario ubuntu.
- UFW aparece activo, con política de entrada deny y salida allow.
- Ollama aparece activo y el modelo qwen2.5:0.5b-instruct figura en la lista de modelos instalados.
- Una petición a la API local generó una respuesta del modelo, aunque limitada por el máximo de tokens configurado.
- El script movió informe.txt a documentos y configuracion.conf, datos.csv y script.sh a trabajo.
- El listado final de archivos confirma cuatro archivos clasificados en las rutas de destino mostradas.

7.2 Matriz de cumplimiento

Actividad	Estado según evidencias	Observación
Estructura de directorios	Evidenciada	Se ven carpetas principales y subcarpetas.
Creación de archivos de prueba	Evidenciada	Cuatro archivos listados en PowerShell.
Transferencia cliente-servidor	Evidenciada tras reintento	Primer intento falló; captura posterior muestra 100 %.
Servicio SSH	Evidenciado	Estado active y conexión remota funcional.
Firewall UFW	Evidenciado	Activo; revisar necesidad de puertos 3000 y 4100.
Instalación/uso de Ollama	Evidenciado parcialmente	Servicio activo, modelo listado y respuesta de API visible.

Clasificación Bash	Evidenciada	Se muestran cuatro movimientos a carpetas destino.
Conteo automático	Requiere revisión	El resumen de salida no parece coincidir claramente con el listado final.
Transferencia de vuelta al cliente	No visible	Las capturas aportadas no muestran descarga del servidor a Windows.
Prueba con todos los tipos de archivo	Parcial	No hay evidencia de PDF, imagen o ZIP reales en la entrada.

8. Seguridad y buenas prácticas

8.1 Protección de SSH y credenciales

La clave privada utilizada para la conexión debe permanecer en el equipo cliente y no incluirse en capturas públicas, repositorios ni anexos del informe. En Windows se recomienda restringir los permisos del archivo de clave a la cuenta propietaria. En el servidor, conviene usar autenticación por clave, mantener actualizado OpenSSH y evitar iniciar sesiones administrativas como root cuando el usuario ubuntu con sudo sea suficiente.

8.2 Reglas del firewall

La política predeterminada de denegar conexiones entrantes es una buena base, pero las excepciones deben responder a una necesidad real. La regla de SSH es necesaria para administrar el servidor de forma remota, aunque idealmente debería limitarse a una IP o red de administración estable. Los puertos 3000 y 4100 aparecen abiertos a Anywhere; antes de conservarlos se debe identificar qué aplicaciones los utilizan, comprobar si requieren exposición pública y considerar restringir su origen.

No debe eliminarse la regla de SSH de una sesión remota activa sin disponer de una vía alternativa de acceso, como la consola de administración del proveedor cloud. Cualquier ajuste del firewall debe realizarse de manera gradual y seguido de una nueva conexión de prueba.

8.3 API de Ollama

La llamada a 127.0.0.1:11434 evita que la prueba dependa de una exposición pública de la API. Se recomienda mantener esa configuración local, no abrir el puerto 11434 en UFW ni en las reglas de red de OCI salvo que exista un caso de uso definido y controles de autenticación y filtrado apropiados. El modelo debe considerarse una herramienta de asistencia; sus propuestas deben revisarse antes de convertirlas en acciones que muevan, borren o modifiquen archivos.

8.4 Seguridad del script

Un clasificador que mueve archivos debe manejar correctamente espacios y caracteres especiales en los nombres, evitar sobrescrituras, registrar cada operación y ofrecer un modo de prueba. Es recomendable conservar una copia de los originales durante el desarrollo y ejecutar el script primero sobre datos de prueba. Para la entrega, conviene que el script compruebe errores y produzca un resumen cuyo conteo se calcule a partir de archivos reales.

8.5 Actualizaciones

La pantalla de bienvenida de Ubuntu informa que hay actualizaciones disponibles y que se requiere reiniciar el sistema. No se debe interpretar esta notificación como prueba de que el sistema esté completamente actualizado. Se recomienda programar las actualizaciones en una ventana de mantenimiento, verificar los servicios después del reinicio y conservar acceso por consola del proveedor para recuperación.

9. Limitaciones y oportunidades de mejora

- La evidencia del primer intento SCP muestra un error de subsistema. Aunque la transferencia posterior se completó, sería conveniente documentar el comando definitivo y explicar la causa del fallo inicial.
- El modelo local produjo una respuesta truncada. Puede aumentarse el límite de salida para una prueba más completa y pedir una tabla clara de extensiones, destinos y precauciones.
- El conjunto de prueba solo incluye cuatro extensiones. Para probar todas las categorías, se recomienda añadir archivos de muestra PDF, JPG/PNG y ZIP, sin usar información sensible.

- El resumen del script debe cotejarse con el listado final. El conteo correcto debe coincidir con los archivos que realmente existen en cada carpeta.
- Las reglas UFW para puertos 3000 y 4100 necesitan una justificación funcional. Si no se requieren desde Internet, deben restringirse a orígenes confiables o eliminarse después de comprobar que no se interrumpe ningún servicio necesario.
- No se observa una transferencia de regreso desde el servidor hacia el cliente Windows. Si ese requisito forma parte de la rúbrica, debe añadirse una captura de descarga con SCP.
- Las capturas no demuestran una prueba de ping ni una transferencia SFTP independiente. Si la guía de la asignatura exige esas actividades expresamente, conviene aportar la evidencia correspondiente.
- El script debe revisarse directamente en el servidor para confirmar sintaxis, manejo de extensiones desconocidas y prevención de sobrescrituras; la captura de código no permite validar cada línea con precisión.

10. Conclusiones

La práctica permitió configurar y utilizar un servidor Ubuntu en la nube desde un cliente Windows, estableciendo un flujo básico de administración remota. La conexión SSH funcionó y el servicio aparece activo, mientras que la transferencia SCP permitió llevar al servidor cuatro archivos creados para la prueba. La verificación de los nombres en la carpeta entrada demuestra que el proceso de copia se completó en un intento posterior al error inicial.

La estructura de directorios proporciona una organización inicial para separar los archivos entrantes, documentos, imágenes, material de trabajo, comprimidos y elementos sin categoría. La automatización mediante Bash reduce el trabajo manual y, en el caso observado, distribuyó los cuatro archivos de acuerdo con las categorías definidas: el archivo TXT quedó en documentos y los archivos CONF, CSV y SH quedaron en trabajo.

La comprobación de Ollama y del modelo qwen2.5:0.5b-instruct demuestra que se pudo ejecutar una consulta de inteligencia artificial local desde la propia instancia. La respuesta obtenida sirve como evidencia de

funcionamiento, aunque su longitud limitada no permite tomarla como una clasificación definitiva ni sustituye la validación humana.

UFW se encontraba activo y aplicaba una política entrante restrictiva por defecto. Aun así, la presencia de reglas para puertos 3000 y 4100 abiertas a cualquier origen hace necesario revisar su justificación. La seguridad de un servidor depende no solo de habilitar el firewall, sino de mantener únicamente las excepciones indispensables y proteger los servicios expuestos.

Como conclusión general, las evidencias documentan un resultado funcional en acceso remoto, transferencia, ejecución de servicios y clasificación automatizada. Para fortalecer la calidad técnica de la entrega, los puntos prioritarios son verificar el resumen del script, ampliar las pruebas de extensiones, documentar la corrección del error SCP y justificar o restringir las reglas adicionales de red.

11. Recomendaciones finales

- Ejecutar nuevamente el script y comparar su resumen con el resultado de find antes de entregar el informe.
- Probar la clasificación con archivos representativos de cada categoría, incluidos PDF, imágenes y ZIP.
- Guardar una copia de seguridad de los archivos de prueba antes de realizar movimientos automáticos.
- Identificar los procesos que escuchan en los puertos 3000 y 4100 y limitar su acceso según el uso real.
- Conservar la API de Ollama ligada a localhost y evitar exponer el puerto 11434 a Internet.
- Actualizar Ubuntu en una ventana controlada y verificar SSH, UFW y Ollama después de reiniciar.
- Añadir una evidencia de transferencia de regreso con SCP si la rúbrica lo solicita.
- Adjuntar el script clasificar.sh como archivo de texto legible, además de la captura, para facilitar su revisión académica.

Anexo A. Comandos de referencia

Los siguientes comandos son referencias para reproducir o validar las operaciones documentadas. Antes de ejecutarlos, compruebe las rutas, el usuario y la dirección de la instancia. No se incluyen claves privadas ni direcciones IP específicas.

A.1 Estructura de directorios

```
mkdir -p /home/ubuntu/parcial-so/{archivos/{entrada,documentos,imagenes,trabajo,comprimidos,otros},scripts,evidencias}
chown -R ubuntu:ubuntu /home/ubuntu/parcial-so
find /home/ubuntu/parcial-so -maxdepth 3 -type d | sort
```

A.2 Verificación de servicios y firewall

```
whoami
hostname
sudo systemctl is-active ssh
sudo systemctl is-active ollama
sudo ufw status verbose
ollama list
```

A.3 Verificación independiente de archivos

```
find /home/ubuntu/parcial-so/archivos -maxdepth 2 -type f -print | sort
find /home/ubuntu/parcial-so/archivos/documentos -maxdepth 1 -type f | wc -l
find /home/ubuntu/parcial-so/archivos/imagenes -maxdepth 1 -type f | wc -l
find /home/ubuntu/parcial-so/archivos/trabajo -maxdepth 1 -type f | wc -l
find /home/ubuntu/parcial-so/archivos/comprimidos -maxdepth 1 -type f | wc -l
find /home/ubuntu/parcial-so/archivos/otros -maxdepth 1 -type f | wc -l
```

Nota: estos conteos son una validación independiente propuesta. Deben ejecutarse en el servidor para obtener los valores reales en el momento de la entrega; no se presentan aquí como resultados ya ejecutados.

Anexo B. Evidencias fotográficas

Se incorporan las nueve capturas del archivo original en el orden recibido. Las leyendas describen la actividad principal visible en cada imagen. Las capturas deben leerse junto con el análisis de los capítulos anteriores.

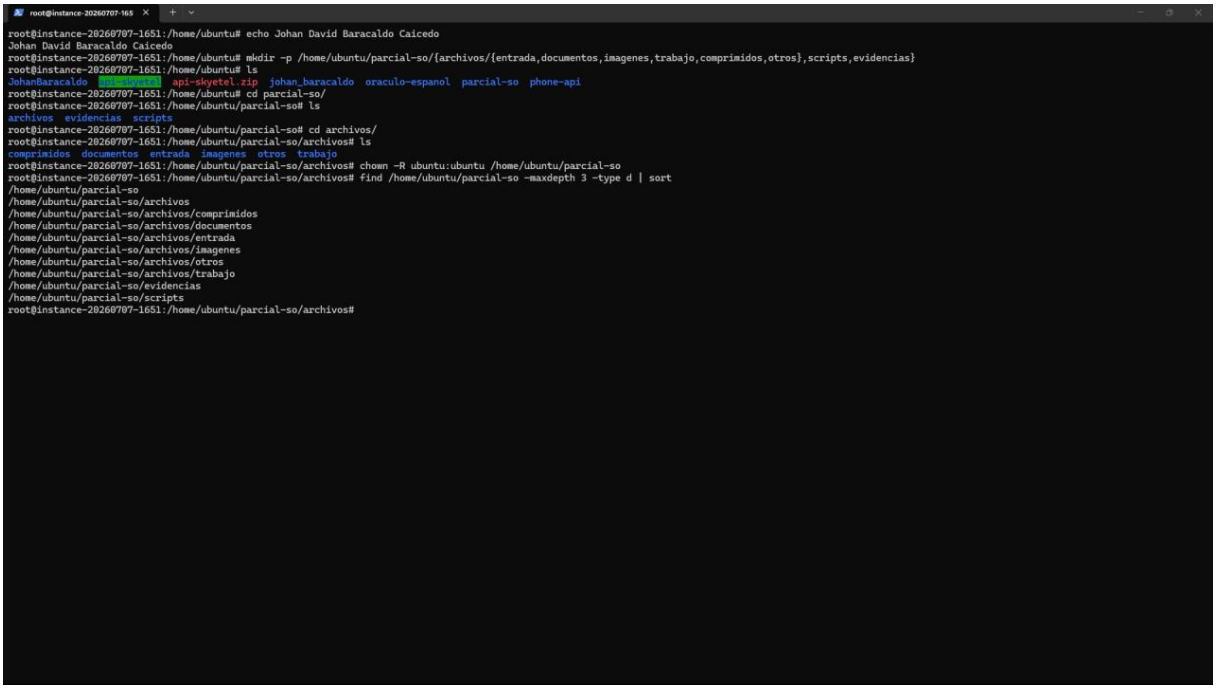


Figura 1. Creación de directorios y permisos en Ubuntu.

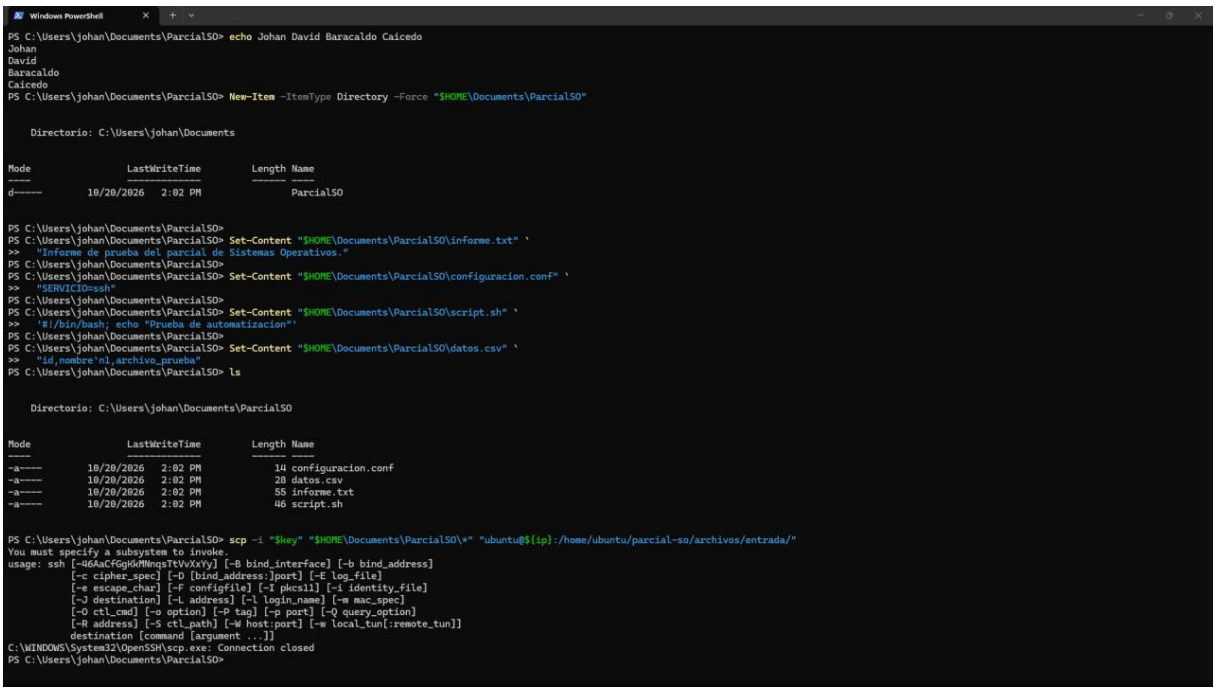


Figura 2. Preparación de archivos de prueba y primer intento SCP.

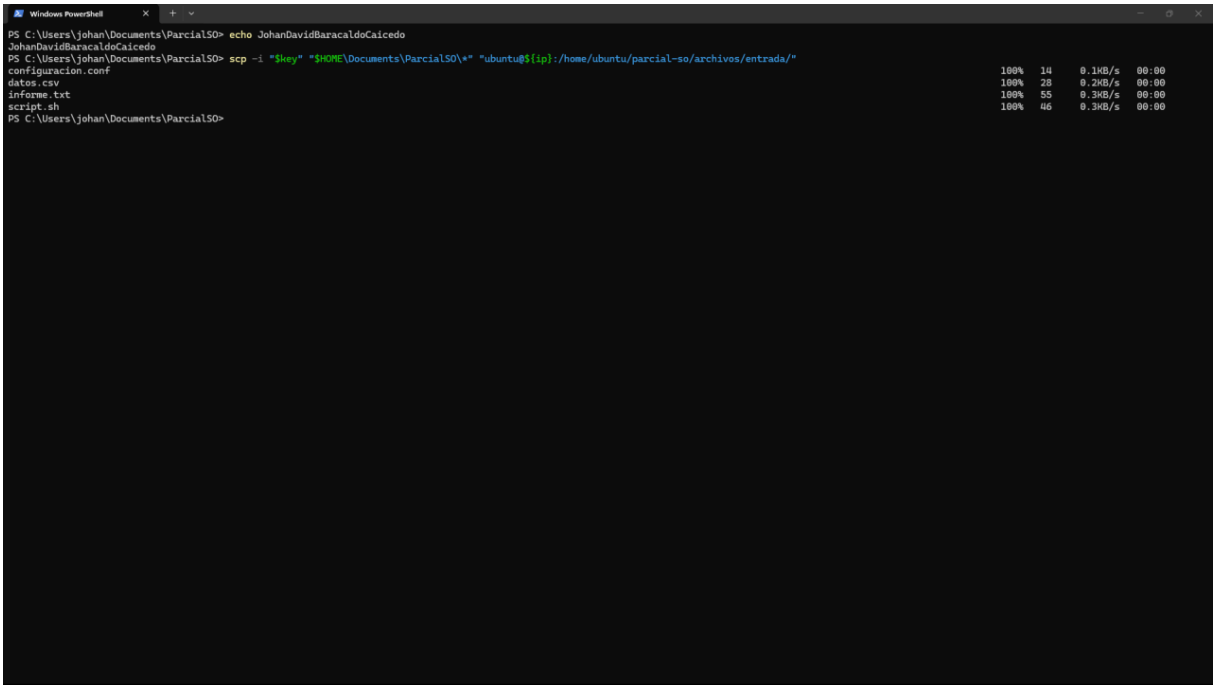


Figura 3. Transferencia de los cuatro archivos desde Windows.

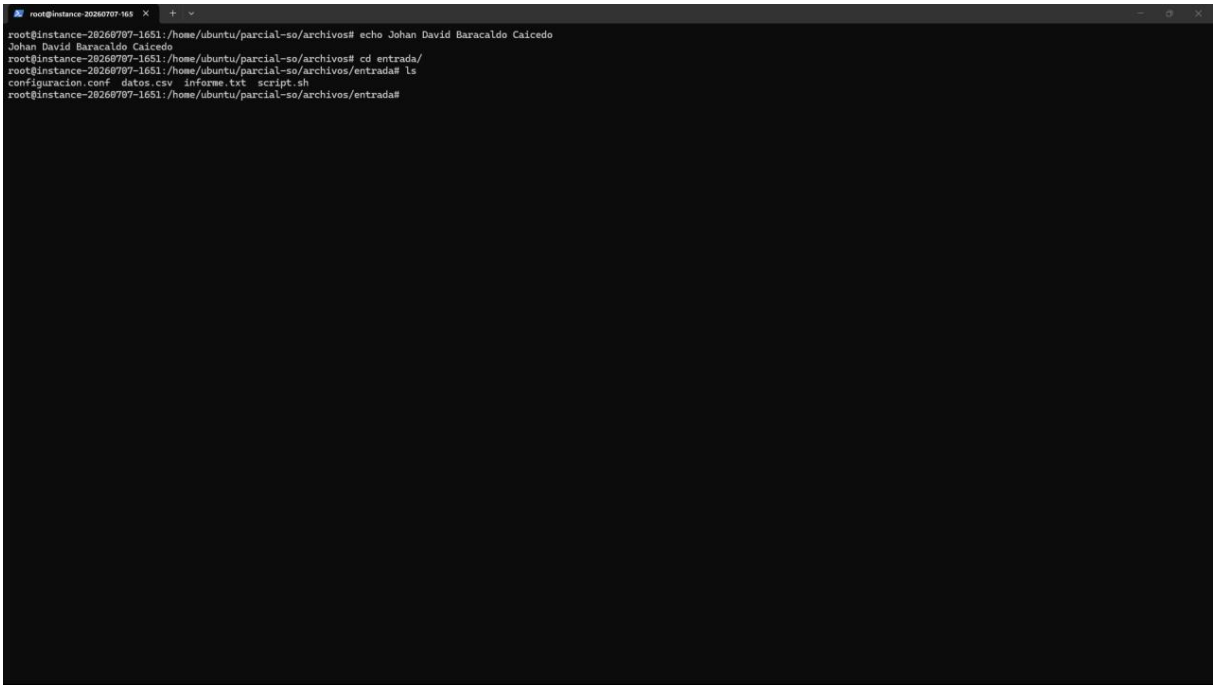


Figura 4. Archivos recibidos en la carpeta de entrada del servidor.

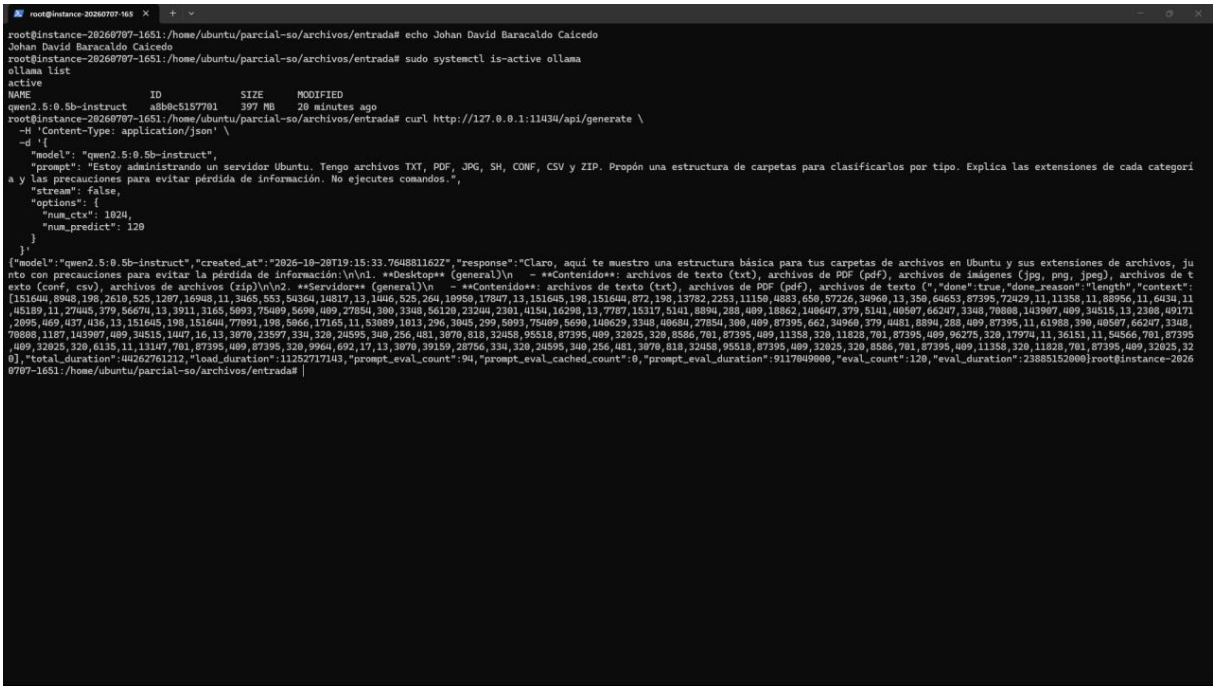
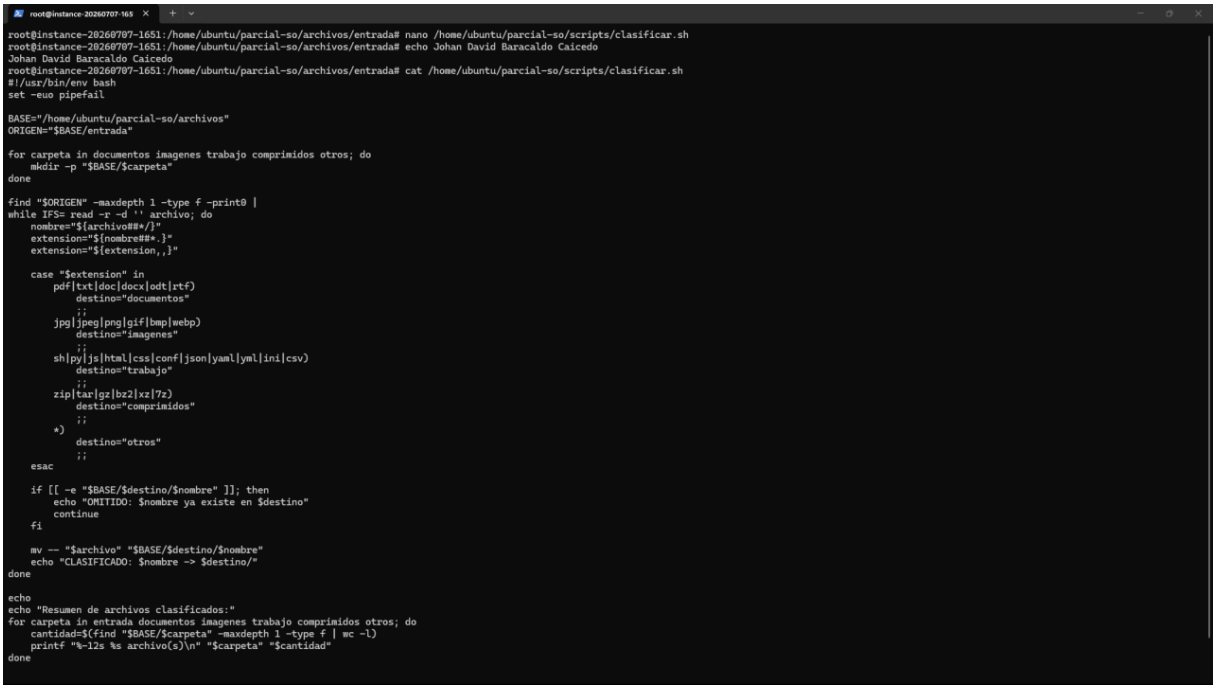


Figura 5. Servicio Ollama, modelo instalado y respuesta de la API local.



```
root@instance-20260707-165
root@instance-20260707-1651:/home/ubuntu/parcial-so/archivos/entrada# nano /home/ubuntu/parcial-so/scripts/clasificar.sh
root@instance-20260707-1651:/home/ubuntu/parcial-so/archivos/entrada# echo Johan David Baracaldo Caicedo
Johan David Baracaldo Caicedo
root@instance-20260707-1651:/home/ubuntu/parcial-so/archivos/entrada# cat /home/ubuntu/parcial-so/scripts/clasificar.sh
#!/usr/bin/env bash
set -euo pipefail

BASE="/home/ubuntu/parcial-so/archivos"
ORIGEN="$BASE/entrada"

for carpeta in documentos imagenes trabajo comprimidos otros; do
    mkdir -p "$BASE/$carpeta"
done

find "$ORIGEN" -maxdepth 1 -type f -print0 |
while IFS= read -r -d '' archivo; do
    nombre="${archivo##*/}"
    extension="${nombre##*.}"
    extension="${extension,,}"

    case "$extension" in
        pdf|txt|doc|docx|odt|rtf)
            destino="documentos"
            ;;
        jpg|jpeg|png|gif|bmp|webp)
            destino="imagenes"
            ;;
        sh|py|js|html|css|conf|json|yaml|yml|ini|csv)
            destino="trabajo"
            ;;
        zip|tar|gz|bz2|xz|7z)
            destino="comprimidos"
            ;;
        *)
            destino="otros"
            ;;
    esac

    if [[ -e "$BASE/$destino/$nombre" ]]; then
        echo "OMITIDO: $nombre ya existe en $destino"
        continue
    fi

    mv -- "$archivo" "$BASE/$destino/$nombre"
    echo "CLASIFICADO: $nombre -> $destino/"
done

echo
echo "Resumen de archivos clasificados:"
for carpeta in entrada documentos imagenes trabajo comprimidos otros; do
    cantidad=$(find "$BASE/$carpeta" -maxdepth 1 -type f | wc -l)
    printf "%-12s %s archivo(s)\n" "$carpeta" "$cantidad"
done
```

Figura 6. Código del script Bash de clasificación.

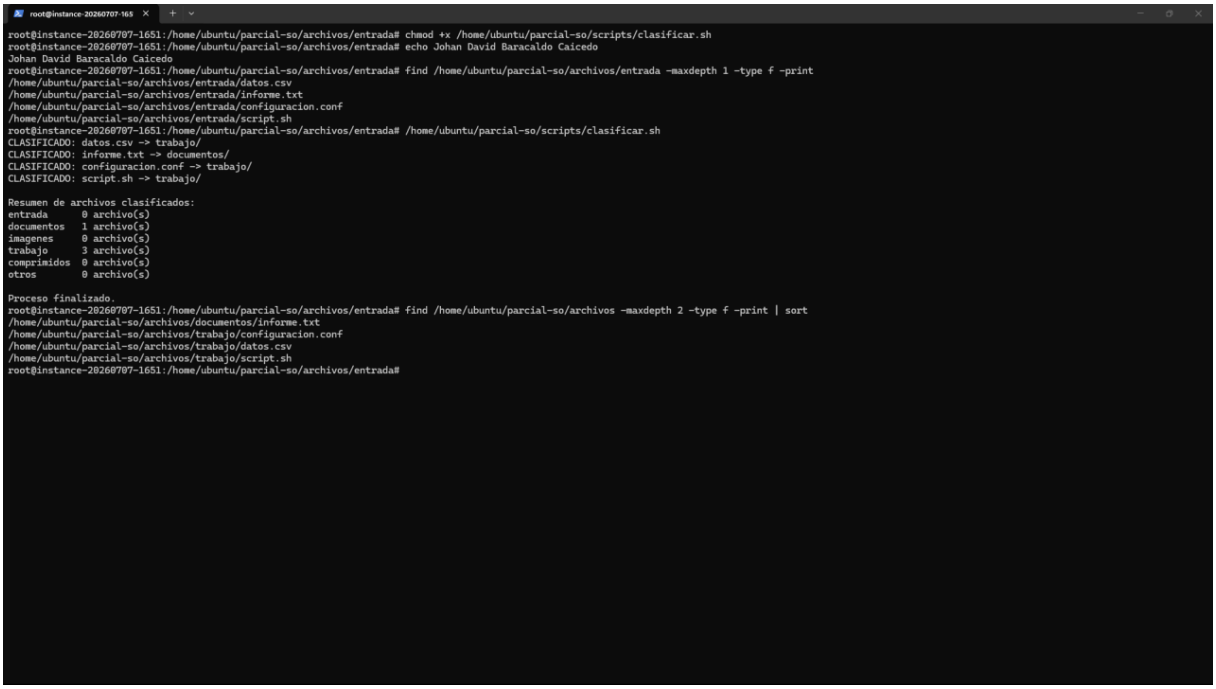


Figura 7. Ejecución del clasificador y verificación de destinos.

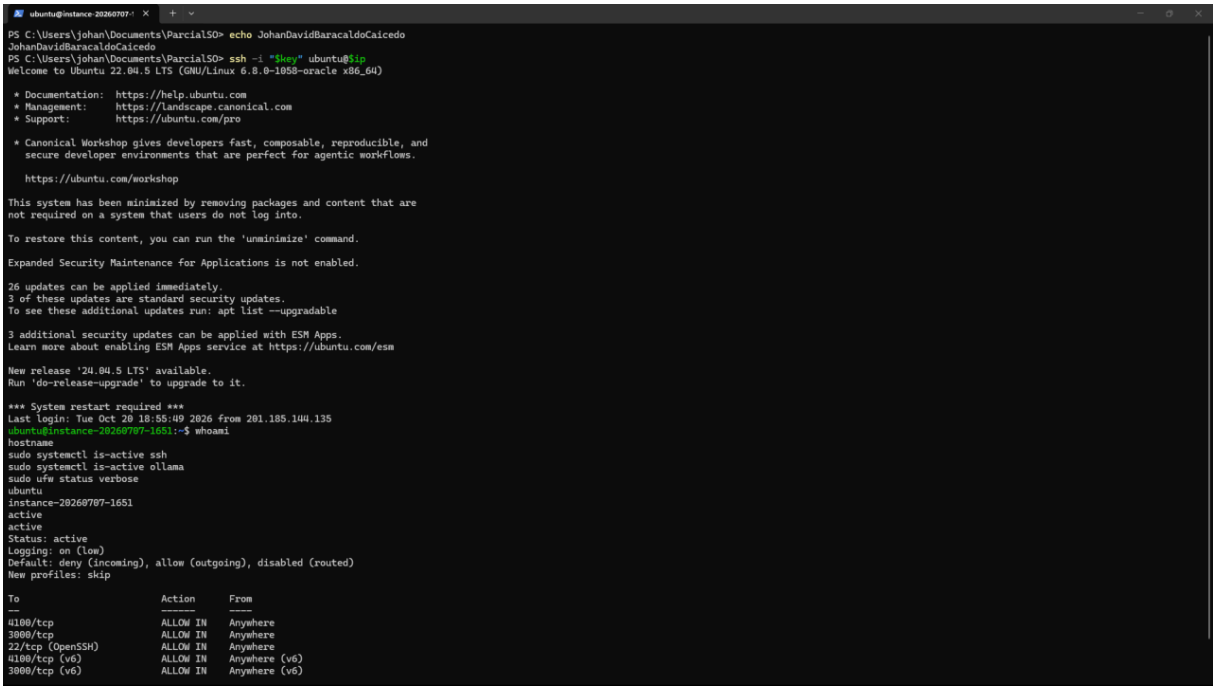


Figura 8. Acceso remoto SSH y estado de los servicios y del firewall.

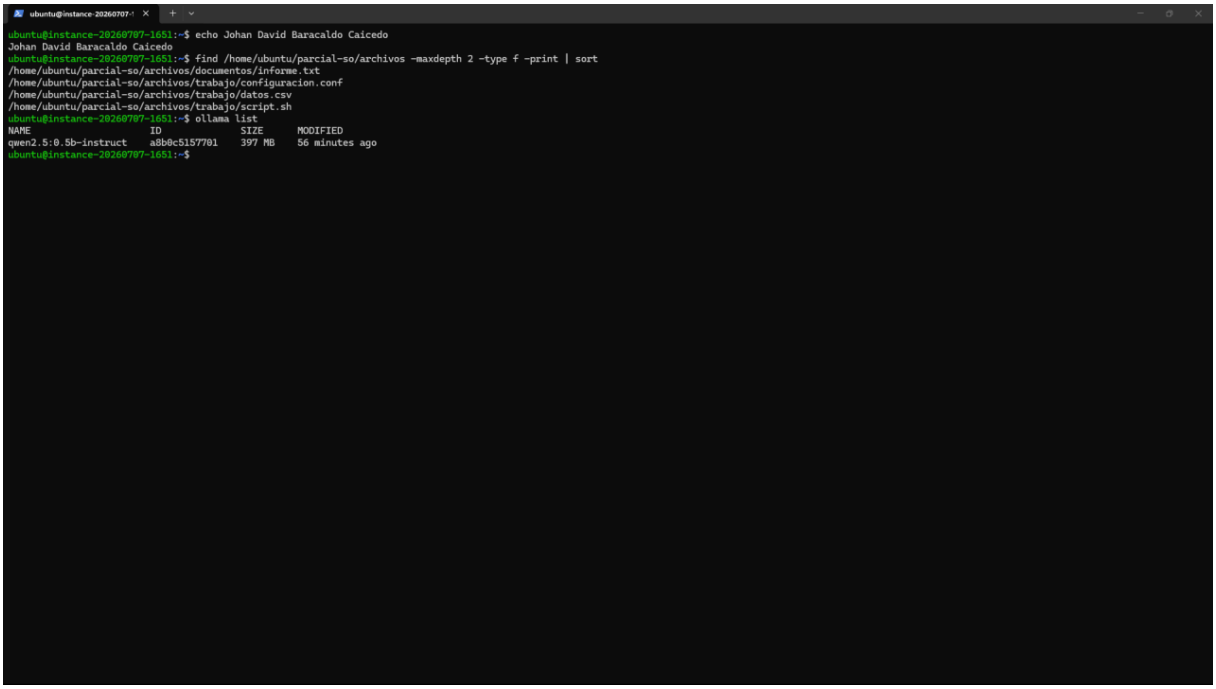


Figura 9. Comprobación final de archivos y del modelo instalado.